DefectLens-A Software Defect Identifier

Software Project Lab 3 Technical Report

Submitted by

Muktadul Islam

BSSE-1215

Institute of Information Technology

University of Dhaka

Supervised by

Dr. Mohammad Shoyaib

Professor

Institute of Information Technology

University of Dhaka

Submission Date: 1 December 2024



Letter of Transmittal

December 1, 2024
BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Technical Report Submission - "**DefectLens**-A Software Defect Identifier"

Assalamualaikum,
I am pleased to submit the technical report for my Software Project Lab 3 project "*DefectLens*-A Software Defect Identifier." This report comprehensively covers the project's technical details, including design, implementation, and testing phases.

The report outlines my innovative approach to automated defect detection using transfer learning techniques and provides detailed documentation of the system's architecture and functionality. While I've tried to do my best, I welcome your feedback for improvement.

Thank you for your consideration.

Sincerely,
Muktadul Islam
Roll: BSSE 1215
Institute of Information Technology
University of Dhaka

Supervisors Signature

Acknowledgement

I am deeply grateful to the Institute of Information Technology, University of Dhaka, for providing me with the opportunity to work on "*DefectLens*-A Software Defect Identifier." I would like to express my sincere gratitude to my supervisor, Dr. Mohammad Shoyaib, for his invaluable guidance and continuous support throughout this project. His expertise and insights have been instrumental in shaping both the direction and implementation of this work.

I would also like to thank my fellow classmates who have provided valuable feedback and suggestions during the development of this project. Their collaborative spirit and constructive input have significantly contributed to its success.

Abstract

DefectLens is an innovative software defect detection tool that addresses the critical challenge of identifying code defects across different software projects efficiently. By leveraging transfer learning techniques and automated metric extraction, *DefectLens* revolutionizes the traditional approach to software quality assurance.

The system automatically extracts key software metrics including McCabe, Halstead, and CK metrics from source code, then employs sophisticated transfer learning models to predict potential defects. What sets *DefectLens* apart is its ability to adapt models trained on one project to accurately detect defects in other projects, effectively addressing the "domain shift" problem that plagues traditional machine learning approaches.

Through its automated analysis capabilities, *DefectLens* significantly reduces the time and effort required for manual code reviews while maintaining high accuracy in defect prediction. The tool offers seamless API integration with existing development workflows and is designed to be scalable across multiple programming languages. This makes it an invaluable asset for software development teams seeking to enhance code quality while optimizing their testing resources.

Table of Contents

1. Introduction....1
1.1 Motivation.... 1
1.2 Scope....1
1.3 Project Timeline....2
1.4 Purpose of Document....2
2. Project Description....3
2.1 Quality Function Deployment (QFD)....3
2.1.1 Normal Requirements....3
2.1.2 Expected Requirements....3
2.1.3 Exciting Requirements....3
3. Scenario Based Modeling....5
3.1 Use Case Diagram.... 5
3.1.1 DefectLens....5
3.1.2 Modules of DefectLens....6
3.1.2.1 Defect Analysis:.... 6
3.1.2.2 Source Code Management:....6
3.1.2.3 Account Management:....6
3.1.3 Modules of Defect Analysis....7
3.1.3.1 Defect Attributes Extraction.... 7
3.1.3.2 History Tracking.... 7
3.1.3.3 Defect Predictions....8
3.1.3.4 Model Training.... 8
3.1.3.5 Transfer Learning Model....8
3.1.4 Modules of Account Management.... 8
3.1.4.1 Login:.... 9
3.1.4.2 Registration:....9
4. Data Based Modeling.... 9
5. Class-Based Modeling....11
5.1 Analysis Classes....11
5.2 Class cards.... 12
5.3 CRC diagram.... 17
6. Architectural Design....18
6.1 Architectural Context Diagram....18
6.2 Define ArcheTypes.... 19
6.3 Refining the Architecture into Components....21
6.3.1 Top Level Components....21
6.3.2 Instantiation of the Architecture.... 22
6.4 Mapping Requirements to Software Architecture.... 23

7. Preliminary Test Plan..... 25
7.1 High Level Description of Testing Goals..... 25
7.2 Test Cases..... 25
Conclusion..... 27

List Of Figures

Figure-1: UseCase Level 0 (DefectLens)..... 5
Figure-2: UseCase Level 1 (Modules of DefectLens)..... 6
Figure-3: UseCase Level 2 (Modules of Defect Analysis)..... 7
Figure-4: UseCase Level 2 (Modules of Account Management)..... 8
Figure-5 : ER-Diagram of DefectLens..... 10
Figure-6: CRC Diagram of DefectLens..... 17
Figure-7: Architectural Context Diagram (ACD)..... 18
Figure-8: Client-Server Architecture Diagram..... 19
Figure-9: Pipe and Filter Architecture Diagram..... 20
Figure-10: Layered Architecture Diagram..... 21
Figure-11: Top Level Components of DefectLens..... 21
Figure-12: Client-Server Architecture of User Interface..... 22
Figure-13: Pipe and Filter Architecture of Source Code Processing..... 23
Figure-14: Pipe and Filter Architecture of Feature Extraction..... 23

List Of Tables

Table-1: Timeline of DefectLens..... 2
Table-2: Schemas of DefectLens..... 11
Table-3: Class Card (User)..... 13
Table-4: Class Card (SourceCodeManager)..... 13
Table-5: Class Card (MetricsExtractor)..... 13
Table-6: Class Card (HalsteadMetrics)..... 14
Table-7: Class Card (McCabeMetrics)..... 14
Table-8: Class Card (CKMetrics)..... 15
Table-9: Class Card (DefectPredictor)..... 15
Table-10: Class Card (SubmissionHistory)..... 16

1. Introduction

Software defect detection is a critical component of the development lifecycle, yet it remains one of the most challenging and time-consuming tasks. *DefectLens* addresses this challenge by introducing an automated approach that combines advanced metric extraction with transfer learning techniques. The tool represents a significant advancement in automated software testing, offering a solution that not only identifies defects efficiently but also adapts its learning across different projects.

DefectLens automatically analyzes source code, extracting critical software metrics such as McCabe, Halstead, and CK metrics. By utilizing transfer learning approaches, it can effectively predict potential defects across different software projects, overcoming the traditional limitations of machine learning models that are restricted to specific datasets.

1.1 Motivation

In modern software development, ensuring code quality is essential but time-consuming, especially when manually identifying defects across multiple projects. Manual code reviews and traditional machine learning models often fall short in detecting defects across projects due to the "domain shift" problem, where models trained on one project perform poorly on others with different data distributions. These limitations require development teams to either invest substantial time in training new models for each project or accept potentially unreliable results.

DefectLens addresses these challenges by employing transfer learning techniques, enabling the tool to adapt models trained on one project to accurately predict defects in another. By automatically extracting and analyzing complex code metrics (Halstead, McCabe, CK metrics), combined with transfer learning capabilities, *DefectLens* provides a robust solution that maintains high accuracy across different projects without requiring extensive retraining. The system's ability to work with both user-provided and predefined training datasets offers flexibility while ensuring consistent performance. This approach significantly reduces the time and resources required for quality assurance while improving the reliability of defect prediction, ultimately leading to better software quality and reduced maintenance costs.

1.2 Scope

The scope of *DefectLens* encompasses:

- **Metric Extraction:** Automated extraction of key software metrics including McCabe, Halstead, and CK metrics.

- ❖ **Transfer Learning Integration:** Implementation of transfer learning techniques for cross-project defect prediction.
- ❖ **Language Support:** Initial focus on Java with planned expansion to C++ and Python.
- ❖ **API Development:** Creation of RESTful APIs for seamless integration with existing development workflows.
- ❖ **Scalability Features:** Design architecture to support multiple programming languages and project types.

1.3 Project Timeline

The development of *DefectLens* will follow this timeline:

Activity	Tentative Timeline
Project Initialization	1st week (Starting from Sept 22, 2024)
Prerequisite Knowledge	2nd-4th week
SRS Documentation & Project Design	5th-7th week
Project Development	8th-15th week
Project Testing & Bug Fixing	15th-18th week
Report Submission	19th week (Final defense on Jan 30, 2025)

Table-1: Timeline of *DefectLens*

1.4 Purpose of Document

This technical document serves several key objectives:

- **Requirements Documentation:** Clearly define and analyze system requirements for *DefectLens*.
- **Architectural Blueprint:** Outline the proposed system architecture and design decisions.
- **Testing Framework:** Establish comprehensive testing methodologies and criteria.
- **Development Guidance:** Provide clear guidelines for implementation and deployment.
- **Knowledge Transfer:** Ensure proper documentation for future maintenance and enhancement.
- **Quality Assurance:** Set standards and metrics for evaluating system performance

The document will serve as a comprehensive guide for development team members, stakeholders, and future maintainers of the *DefectLens* system.

2. Project Description

DefectLens is an innovative software testing tool that revolutionizes the defect detection process through automation and intelligent analysis. By combining sophisticated metric extraction capabilities with transfer learning techniques, the system offers a comprehensive solution for identifying potential software defects across different projects. The tool addresses the critical gap in current software testing approaches by enabling efficient cross-project defect prediction while minimizing the need for extensive manual testing.

2.1 Quality Function Deployment (QFD)

QFD is a technique that translates customer needs into technical requirements for the software. With respect to *DefectLens*, the following requirements are identified:

2.1.1 Normal Requirements

1. **Metric Extraction Functionality:** *DefectLens* should successfully extract software metrics (McCabe, Halstead, CK) from source code files using automated parsers and analysis techniques.
2. **Defect Prediction:** The system must accurately predict potential defects by analyzing extracted metrics using transfer learning models trained on labeled datasets.
3. **Source Code Upload and Processing:** The tool should take source code and process source code efficiently with minimal resources.
4. **Basic Interface:** Users should be able to easily upload code files and view analysis results through a simple, intuitive interface.
5. **Exports Defect Result:** User should be able to download the final result created by the system over the source code.

2.1.2 Expected Requirements

1. **Model Adaptability:** *DefectLens's* machine learning models should demonstrate high accuracy in defect prediction across different projects.
2. **API Support:** The system should provide RESTful APIs for seamless integration with existing development workflows.
3. **Result Analysis:** Comprehensive reports detailing detected defects, their locations, and severity levels should be available.

2.1.3 Exciting Requirements

1. **Multi-Language Support:** The system should extend support beyond Java to include C++ and Python codebases.

2. **Advanced Visualizations:** Interactive dashboards for defect visualization and trend analysis.

2.2 Usage Scenario

The system will be a web-based tool that will help users identify potential defects in their source code using advanced metrics analysis and transfer learning techniques. *DefectLens* will provide detailed analysis of the defects it detects and give comprehensive reports to the users. Usage scenario for the tool can be segmented as follows:

Source Code Upload

Users can upload their source code in zip format through the intuitive web interface. The system will also support both training data uploads for customs modeling and target/source code for defect prediction.

Real-time Metrics Extraction and Analysis

DefectLens will automatically process the uploaded source code to extract critical software metrics (Halstead, McCabe, CK). It will leverage transfer learning techniques based on existing defect patterns to analyze new code effectively. The system processes these metrics through its machine learning models to identify potential defects in files or modules.

Comprehensive Analysis Report

Upon completing the analysis, DefectLens will generate a detailed report highlighting potential defects in the code. Users can explore an interactive visualization.

Model Accuracy Verification

The system will provide comprehensive tools to evaluate model performance. Users can view detailed accuracy metrics, compare predictions against known defects, and analyze performance across multiple projects.

History and Progress Tracking

The system ensures a record of user interactions with the logs submission records with unique IDs, generates file hash values for data verification, and stores historical analysis reports. Users can track progress and monitor improvements in defect detection over time.

3. Scenario Based Modeling

Scenario-based modeling is an approach used in software development and design to create models that depict how a system behaves or reacts in various real-world situations or scenarios. These scenarios help developers understand and visualize how the software will function in different contexts.

3.1 Use Case Diagram

A use case diagram is a visual representation in software engineering that illustrates how a system interacts with its external entities or actors to achieve specific goals or tasks. It shows the relationships between use cases (representing system functionalities) and actors (representing external users or systems). Use case diagrams are typically created through requirements gathering, where system functionalities and user interactions are identified, and then diagrammed using specialized software modeling tools.

3.1.1 DefectLens

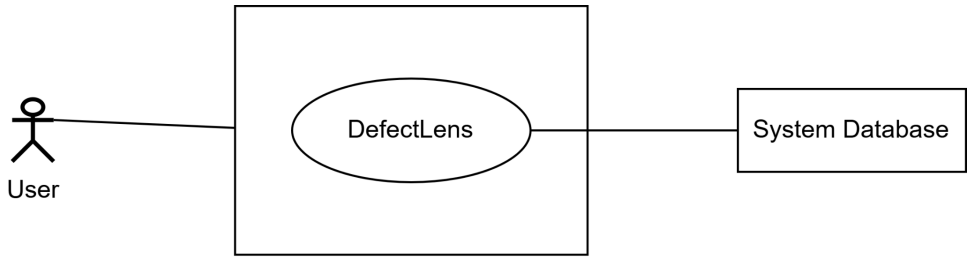


Figure-1: UseCase Level 0 (*DefectLens*)

Primary actors: User
Secondary actors: System Database
Goal in Context: The diagram represents the high-level interactions between the User and the *DefectLens* system for automated software defect detection using transfer learning techniques.

3.1.2 Modules of DefectLens

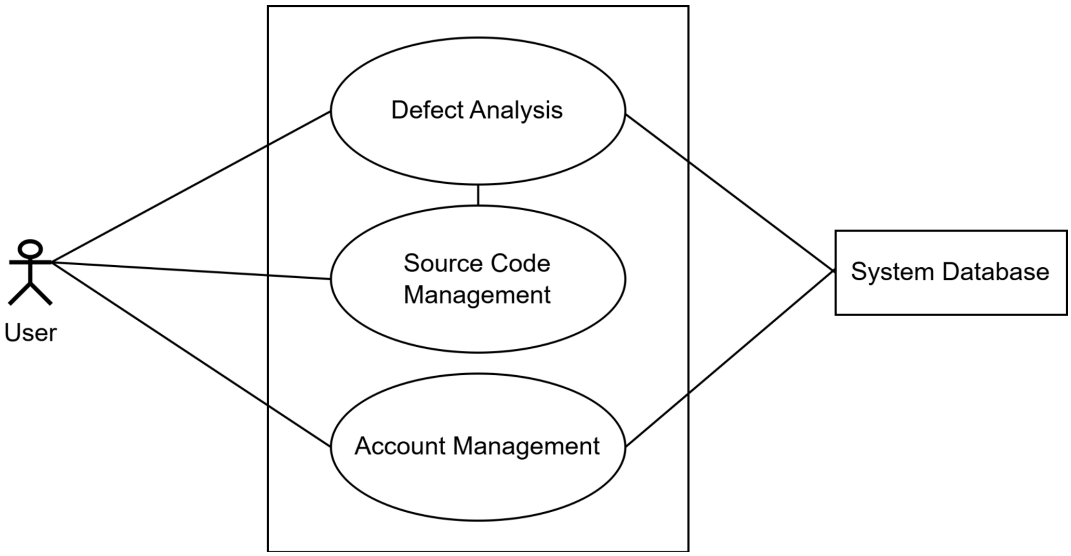


Figure-2: UseCase Level 1 (Modules of *DefectLens*)

Primary actors: User
Secondary actors: System Database
Goal in Context: This level provides an overview of the main modules in the *DefectLens* system, encompassing user account management, source code handling, and the core defect analysis functionality. The modules can be elaborated as follows,

3.1.2.1 Defect Analysis:

The "Defect Analysis" module forms the core of *DefectLens's* functionality. It encompasses several sub-modules that work together to analyze source code, predict defects, and generate insightful reports.

3.1.2.2 Source Code Management:

The "Source Code Management" module facilitates the uploading, storage, and management of user project source code. It allows users to provide the codebase they want to analyze for potential defects. This module interacts with the system database to securely store and retrieve user-specific source code files.

3.1.2.3 Account Management:

The "Account Management" module serves as the entry point for *DefectLens's* user management functionality. It handles user authentication and account-related tasks, ensuring secure access to the system's features. This module encompasses two sub-modules: Login and Registration.

3.1.3 Modules of Defect Analysis

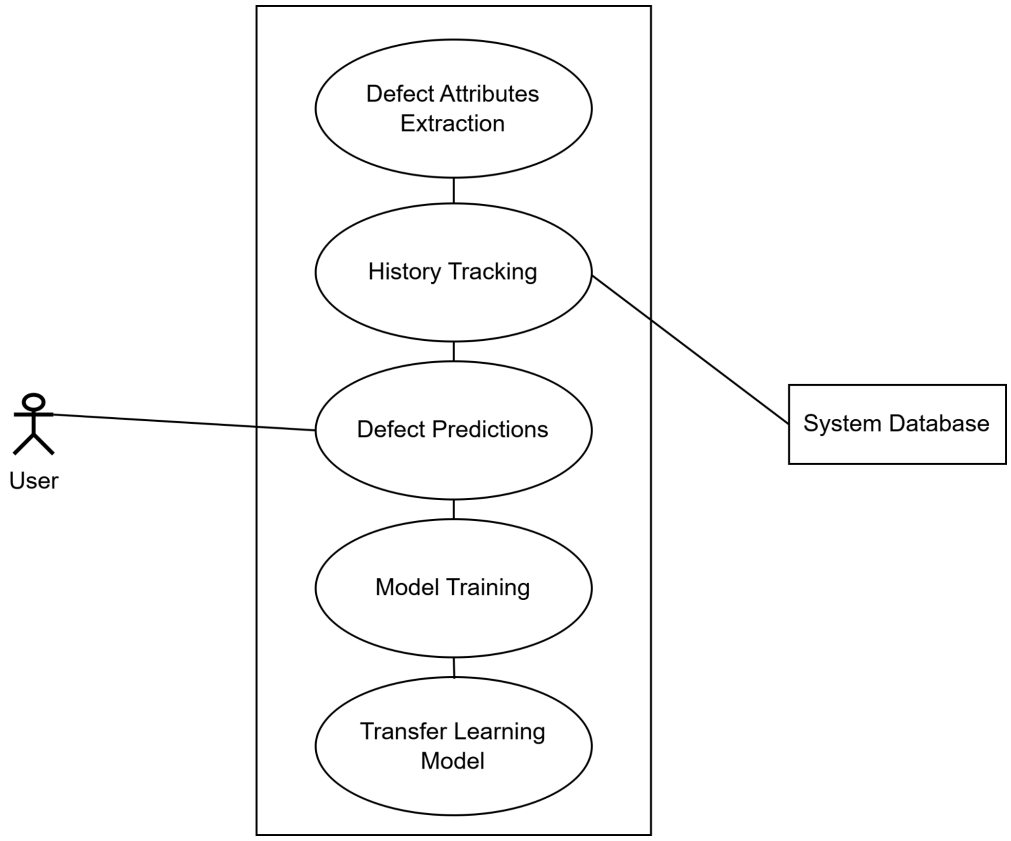


Figure-3: UseCase Level 2 (Modules of Defect Analysis)

Primary actors: User
Secondary actors: System Database
Goal in Context: The sub-modules in this level work together to perform comprehensive defect analysis on user-provided source code, leveraging advanced techniques such as transfer learning. The modules can be elaborated as follows,

3.1.3.1 Defect Attributes Extraction

The "Defect Attributes Extraction" sub-module preprocesses the uploaded source code and extracts relevant code metrics and attributes. It performs static code analysis to gather essential features that serve as input for the defect prediction models.

3.1.3.2 History Tracking

The "History Tracking" sub-module maintains a record of all the defect prediction analyses performed by the user. It stores analysis metadata, including timestamps, source code versions, and prediction results, allowing users to reference and compare past analyses.

3.1.3.3 Defect Predictions

The "Defect Predictions" sub-module utilizes trained machine learning models to predict potential defects in the uploaded source code. It applies the extracted code attributes to the models and generates predictions, identifying code segments that are likely to contain defects.

3.1.3.4 Model Training

The "Model Training" sub-module enables users to train custom defect prediction models using their own labeled datasets. It provides an interface for users to upload labeled source code examples, trains the models based on the provided data, and stores the trained models for future use.

3.1.3.5 Transfer Learning Model

The "Transfer Learning Model" sub-module enhances *DefectLens's* prediction capabilities by leveraging knowledge transfer from models trained on different but related codebases. It allows the system to adapt and improve its defect prediction accuracy by incorporating insights from pre-trained models.

3.1.4 Modules of Account Management

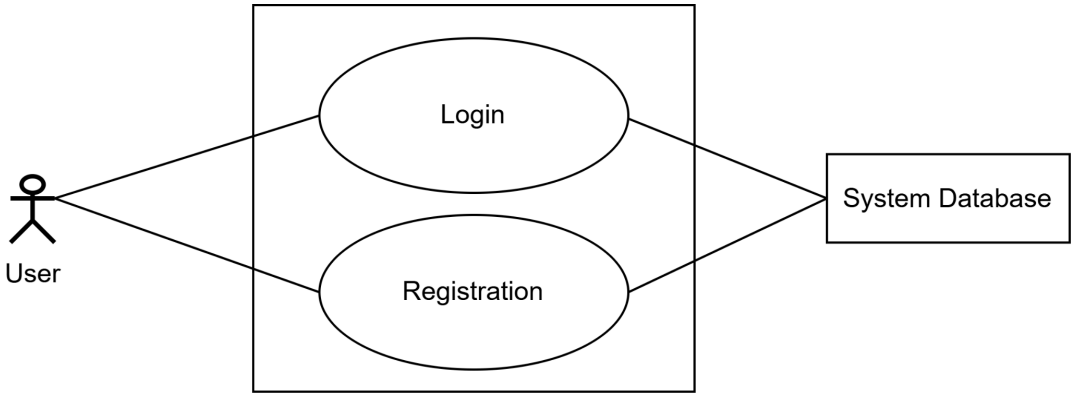


Figure-4: UseCase Level 2 (Modules of Account Management)

Primary actors: User
Secondary actors: System Database
Goal in Context:: The sub-modules in this level handle user authentication and account creation, ensuring secure access to the *DefectLens* system. The modules can be elaborated as follows,

3.1.4.1 Login:

The "Login" sub-module allows registered users to securely log into their *DefectLens* accounts. It verifies user credentials, authenticates their identity, and grants access to the system's personalized features and data.

3.1.4.2 Registration:

The "Registration" sub-module enables new users to create a *DefectLens* account. It collects necessary user information, validates the data, and securely stores it in the system database, granting the user access to the *DefectLens* platform.

4. Data Based Modeling

Database modeling is a visual representation of a database structure and how data is organized within it. It involves creating entity-relationship diagrams (ERDs) that depict tables (entities), attributes (columns), and the relationships between them. Database modeling is typically achieved by analyzing the data requirements of a system, identifying entities and their attributes, and then using modeling tools or software to create ERDs that serve as a blueprint for database design and implementation.

The entities of *DefectLens* are analyzing the usage story are:

- Users
- Projects
- ProjectMetricsData
- ProjectPredictionData
- Models
- SourceFile

The ER-Diagram of *DefectLens* can be shown as below:

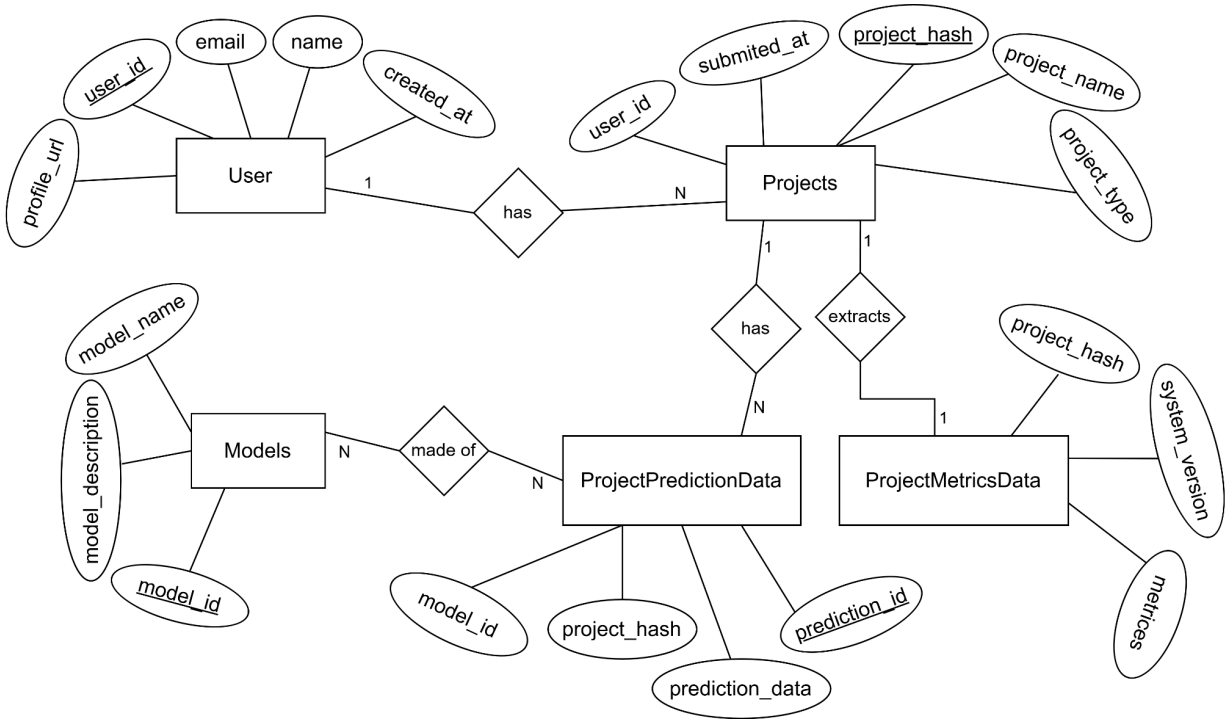


Figure-5 : ER-Diagram of *DefectLens*

The schemas derived from the ER-diagram is as follows:

Schema	Attributes	Type
Users	user_id (PK)	varchar(100)
	email	varchar(100)
	name	varchar(100)
	profile_url	varchar(250)
	created_at	datetime
Models	model_id (PK)	varchar(50)
	model_name	varchar(50)
	model_descriptions	varchar(500)

Projects	project_hash (PK)	varchar(65)
	user_id (FK)	varchar(100)
	project_name	varchar(100)
	submited_at	datetime
	project_type	varchar(20)
ProjectMetricsData	project_hash (FK)	varchar(65)
	system_version	varchar(15)
	metrices	Objects
ProjectPredictionData	prediction_id (PK)	varchar(50)
	project_hash (FK)	varchar(65)
	model_id (FK)	varchar(50)
	prediction_data	Objects

Table-2: Schemas of *DefectLens*

5. Class-Based Modeling

Class-based modeling is crucial in designing *DefectLens's* object-oriented architecture. The system's complexity requires careful structuring of classes to handle source code analysis, metrics extraction, and defect prediction effectively.

5.1 Analysis Classes

After analyzing the requirements using General Classification criteria (External entities, Things, Events, Roles, Organizational units, Places, Structures) and Selection Criteria (Retained information, Needed services, Multiple attributes, Common operations, Essential requirements), the following core classes emerged:

- User
- SourceCodeManager
- HalsteadMetrics
- McCabeMetrics

- CKMetrics
- DefectPredictor
- SubmissionHistory
- FileProcessor

Upon applying software design principles, it was observed that the FileProcessor class represents a specific behavior or operational responsibility closely aligned with the SourceCodeManager. To adhere to the Single Responsibility Principle (SRP) and reduce unnecessary fragmentation of functionality, the behavior of FileProcessor was integrated into the SourceCodeManager class. This consolidation streamlines the design and improves cohesion within the system.

Moreover all the defect metrics extractor classes such as HalsteadMetrics, McCabeMetrics, CKMetrics have the same kind of responsibilities, we can follow Strategy Pattern and Dependency Injection in it, and in order to implement such, we can derive a generic classier class.

The final set of refined classes is:

- User
- SourceCodeManager
- MetricsExtractor
- HalsteadMetrics
- McCabeMetrics
- CKMetrics
- DefectPredictor
- SubmissionHistory

5.2 Class cards

The class cards of *DefectLens* is given below:

User	
Attributes	**Methods**
- userId - username - email - profile_url - signup_at	+ register() + login() + getter() + setter() + viewSubmissionHistory()

	+uploadSourceCode() +uploadSourceDataSet() +uploadTargetDataSet()
Responsibilities	**Collaborators**
• Manage user account • Control source code submissions • Track submission history	SubmissionHistory, DefectPredictor

Table-3: Class Card (User)

SourceCodeManager	
Attributes	**Methods**
- sourceId - fileName - uploadDate - status - fileContent	+ getter() + validateZipFile(file) + extractFiles() + computeFileHash() + processSourceCode() + getSourceMetrics()
Responsibilities	**Collaborators**
• Handle source code uploads • Extract code content • Initiate metrics extraction	DefectPredictor

Table-4: Class Card (SourceCodeManager)

MetricsExtractor	
Attributes	**Methods**
- metricsId - sourceCode - metrics	+ getter() + extractAllMetrics() + aggregateMetrics()
Responsibilities	**Collaborators**
• Coordinate metrics extraction • Aggregate all metrics	DefectPredictor

Table-5: Class Card (MetricsExtractor)

HalsteadMetrics	
Attributes	**Methods**
- unique_operators - unique_operands - total_operators - total_operands	- setter() + getter() + extractAllMetrics() + aggregateMetrics() - get_unique_operators() - get_unique_operands() - get_total_operators() - get_total_operands()
Responsibilities	**Collaborators**
● Calculate Halstead metrics ● Identify operators/operands ● Compute complexity measures	DefectPredictor, MetricsExtractor

Table-6: Class Card (HalsteadMetrics)

McCabeMetrics	
Attributes	**Methods**
- cycloComplexity - edges - nodes - controlFlowGraph	- setter() + getter() + extractAllMetrics() + aggregateMetrics() - calculateComplexity() - analyzeControlFlow() - buildControlFlowGraph() - countDecisionPoints()
Responsibilities	**Collaborators**
● Calculate cyclomatic complexity ● Analyze control flow ● Track decision points	DefectPredictor, MetricsExtractor

Table-7: Class Card (McCabeMetrics)

CKMetrics	
Attributes	**Methods**
- weightedMethods - numberOfChildren - couplingBetweenObjects - responseForClass - lackOfCohesion	- setter() + getter() + extractAllMetrics() + aggregateMetrics() - calculateWMC() - calculateNOC() - calculateCBO() - calculateRFC() - calculateLCOM()
Responsibilities	**Collaborators**
• Calculate CK metrics suite • Analyze class relationships • Measure code quality	DefectPredictor, MetricsExtractor

Table-8: Class Card (CKMetrics)

DefectPredictor	
Attributes	**Methods**
- modelId - modelName - sourceMetrics - trainingData - predictions - confidenceScores	+ setter() + getter() + getAllMatrices() + trainModel() + transferKnowledge() + predictDefects() + evaluateAccuracy()
Responsibilities	**Collaborators**
• Train prediction models • Apply transfer learning • Generate defect predictions • Evaluate model performance	MetricsExtractor, SourceCodeManager, SubmissionHistory, SubmissionHistory, User

Table-9: Class Card (DefectPredictor)

SubmissionHistory	
Attributes	**Methods**
- submissionId - userId - sourceId - timestamp - results - status	+ setter() + getter() + recordSubmission() + updateStatus() + getSubmissionDetails() + generateReport() + exportHistory()
Responsibilities	**Collaborators**
● Track submissions ● Store analysis results ● Generate reports ● Maintain history	User, DefectPredictor

Table-10: Class Card (SubmissionHistory)

5.3 CRC diagram

The CRC diagram of the classes can is given below:

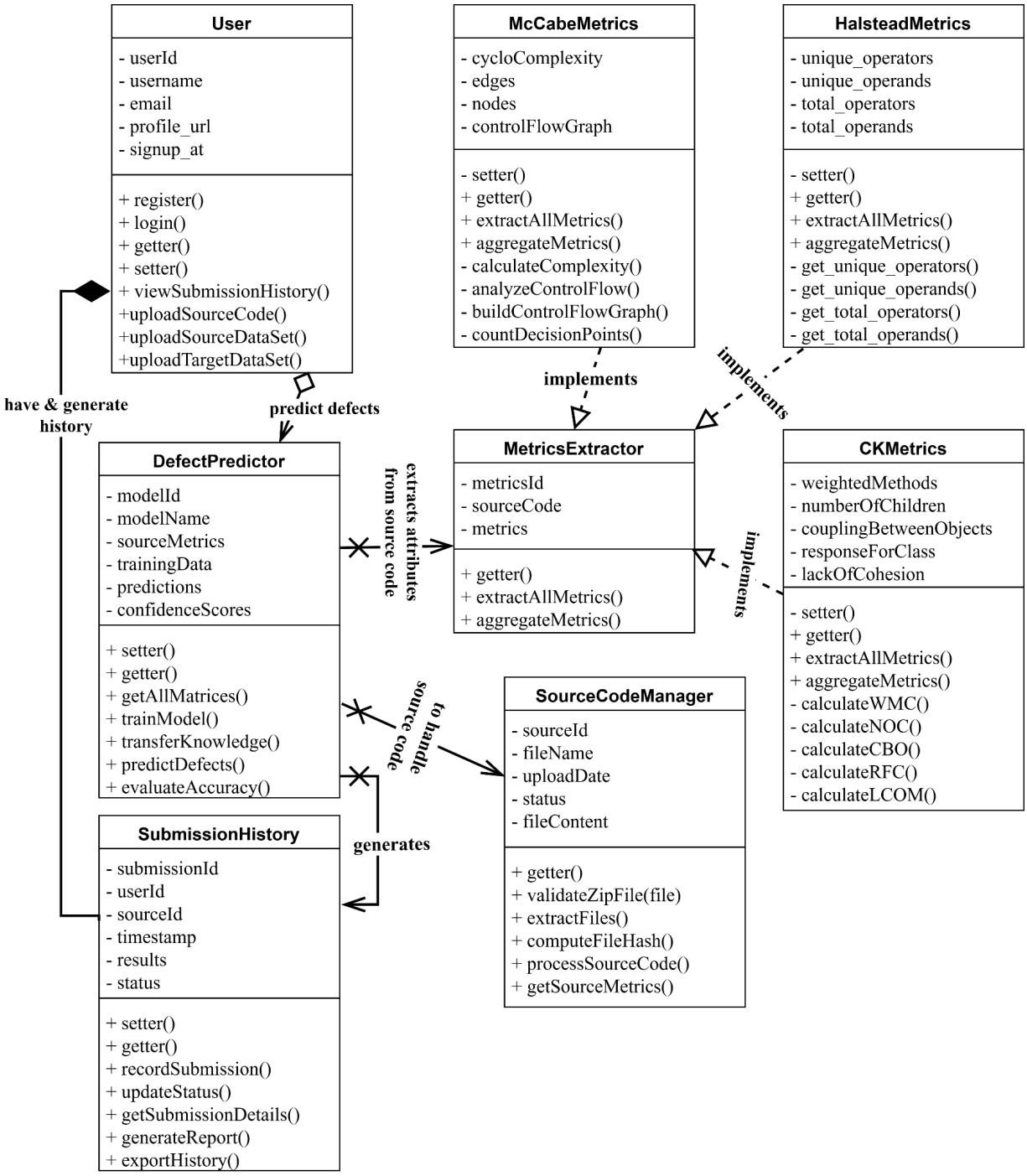


Figure-6: CRC Diagram of DefectLens

6. Architectural Design

Architectural design is a visual representation in software engineering that outlines the high-level structure and organization of a software system. It focuses on denying the major components or modules of the system, their interactions, and the overall system's architecture.

6.1 Architectural Context Diagram

At the architectural design level, an architectural context diagram (ACD) models how software interacts with external entities, including superordinate, subordinate, and peer-level systems, and actors, all communicating with the target system through interfaces.

Representation of *DefectLens* in Architectural Context Diagram(ACD) :

Superordinate systems - None
Subordinate systems - AST Engine, Google Authentication
Peer-level systems - None
Actors - Developers, Researchers

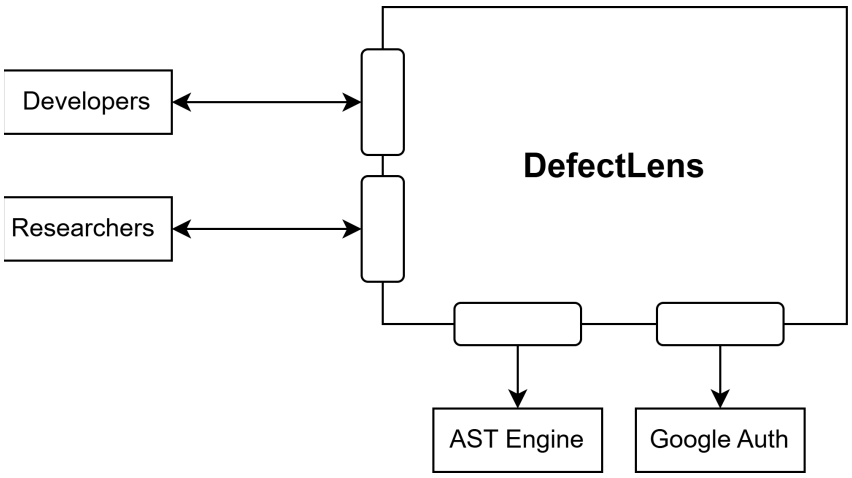


Figure-7: Architectural Context Diagram (ACD)

6.2 Define ArcheTypes

An archetype is a class or pattern that represents a core abstraction that is critical to the design of an architecture for the target system. In general, relatively small set archetypes are required to design even relatively complex systems. The target system architecture is composed of these archetypes, which represent stable elements of the architecture but may be instantiated in many different ways based on the behavior of the system. In many cases, archetypes can be derived by examining the analysis classes defined as part of the requirements model. In our cases, the archetypes are:

- Client-Server Architecture
- Pipe and Filter Architecture
- Layered (N-Tier) Architecture

Client-Server Architecture

Client-server architecture is a cornerstone of modern system design, where the network infrastructure is structured to include multiple clients and a central server. In this model, clients are devices or programs that make requests for services or resources, while the server is a powerful machine or software that fulfills these requests. Communication between clients and the server follows a request-response protocol, such as HTTP/HTTPS for web services or SQL for database queries.

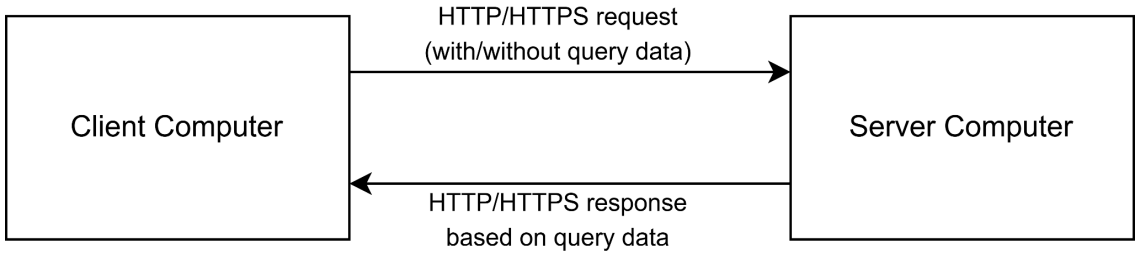


Figure-8: Client-Server Architecture Diagram

Pipe and Filter Architecture

The Pipe and Filter architecture is a design pattern that organizes a process into a series of modular steps, referred to as filters, connected by pipes. This approach is particularly useful for systems that require a sequence of operations, like data processing, where each step performs a specific function.

Filters: Each filter handles one specific task, like transforming, validating, or processing data. For example, the first filter might process raw data and pass it to the next filter, which does further processing.

Pipes: Pipes are the connections that move data between filters, ensuring that the output from one becomes the input for the next. In diagrams, pipes are shown as arrows connecting the filters.

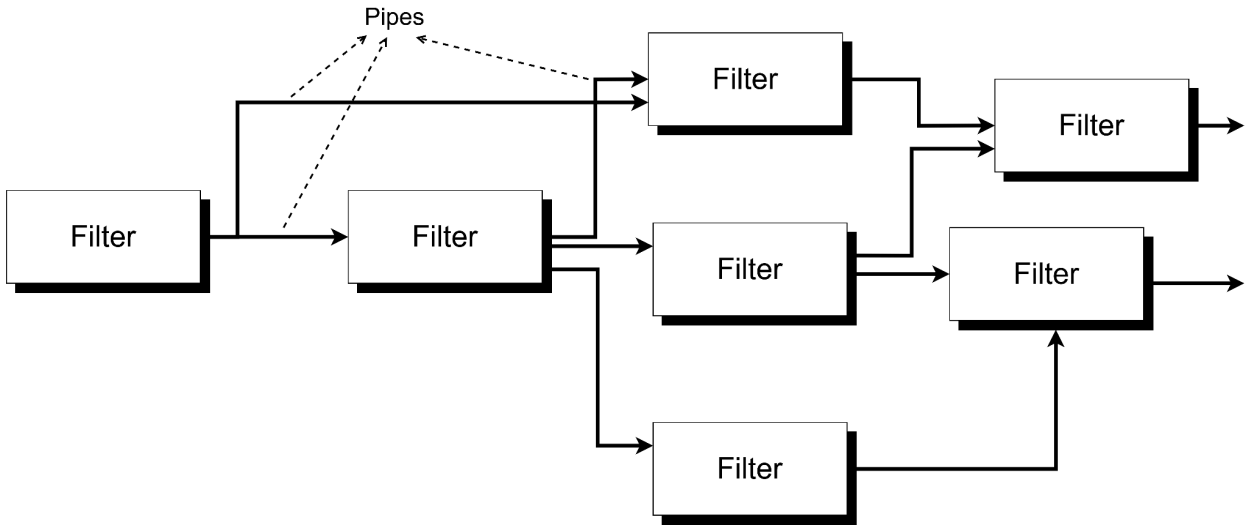


Figure-9: Pipe and Filter Architecture Diagram

Layered (N-Tier) Architecture

The Layered Architecture organizes a system into separate layers, where each layer focuses on a specific responsibility. This structure makes the application modular and easier to manage. Layers only interact with the one above or below them.

Presentation Layer (UI): The part the user interacts with. It handles input and displays output.
Application Layer (Business Logic): Manages the main logic of the system, like processing requests and applying business rules.
Utility Layer (Database): Responsible for retrieving and storing data from the database, keeping data-related tasks separate.

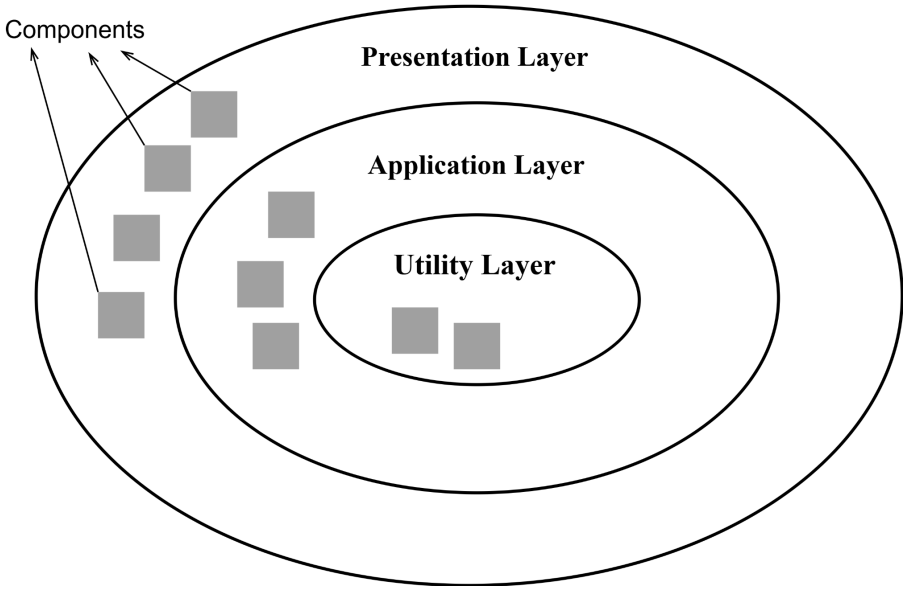


Figure-10: Layered Architecture Diagram

6.3 Refining the Architecture into Components

6.3.1 Top Level Components

Overall architectural structure with top level components is illustrated below-

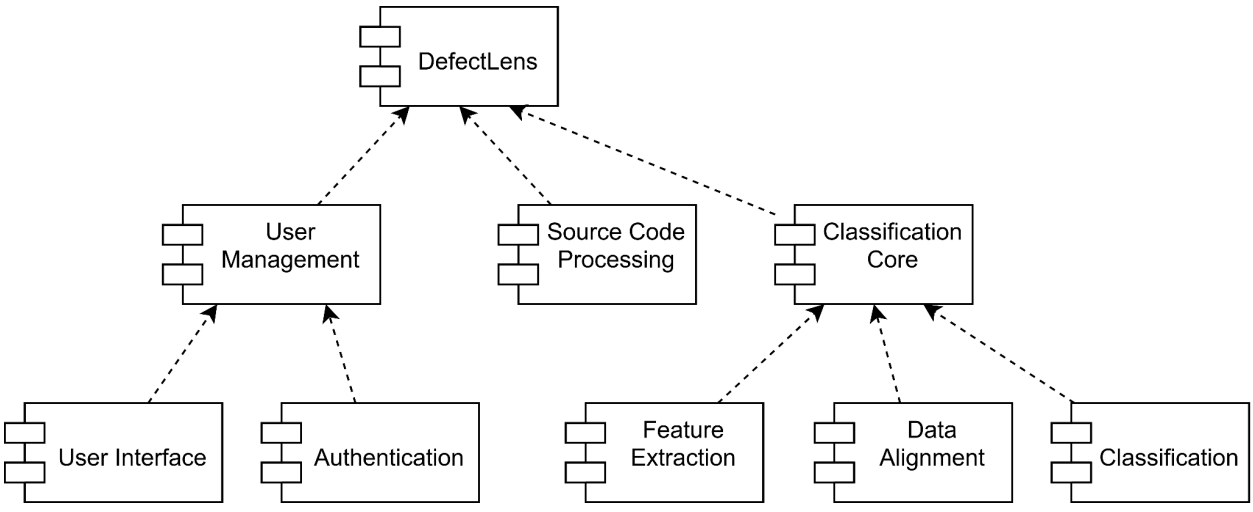


Figure-11: Top Level Components of *DefectLens*

6.3.2 Instantiation of the Architecture

Component: User Interface
Pattern: Client-Server Architecture

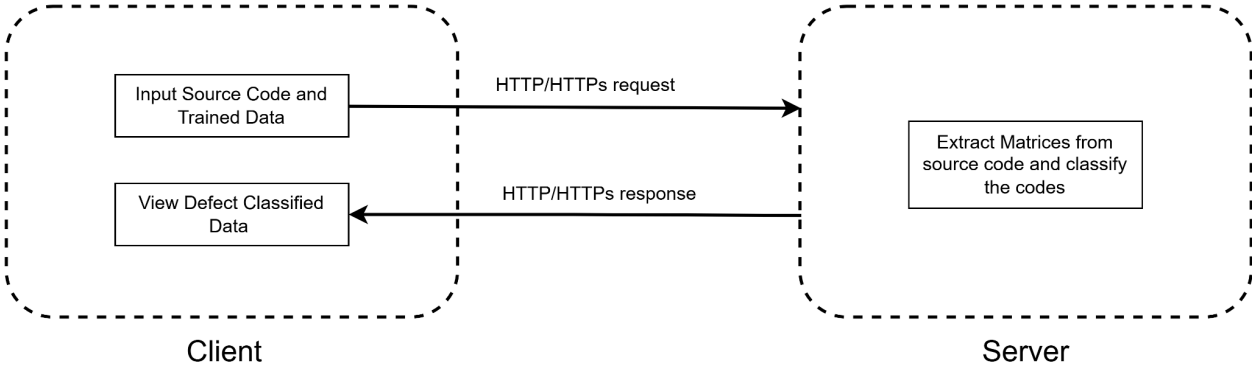


Figure-12: Client-Server Architecture of User Interface

Component: Authentication
Pattern: Layered (N-Tier) Architecture

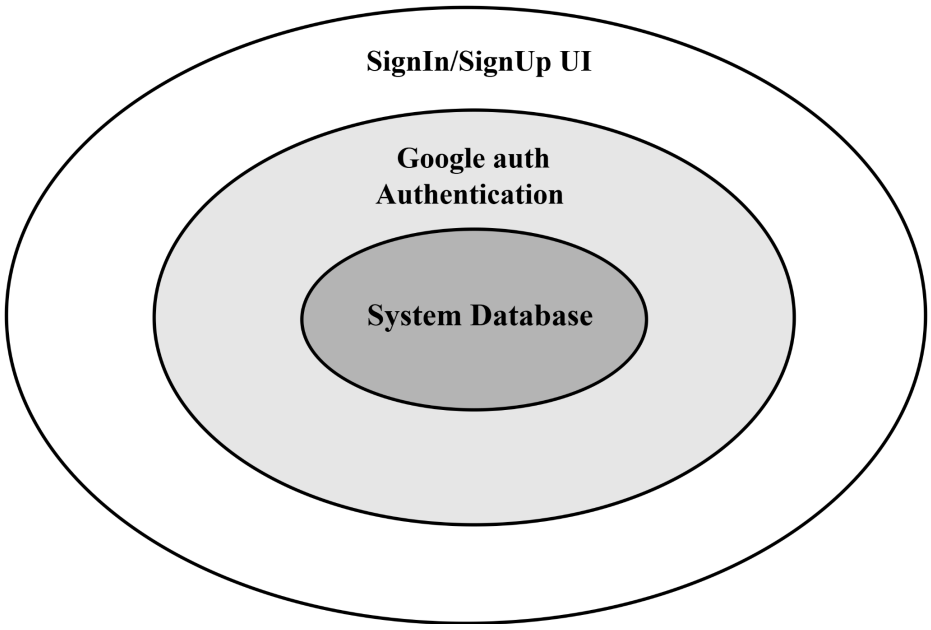


Figure-12: Client-Server Architecture of User Interface

Component: Source Code Processing
Pattern: Pipe and Filter Architecture



Figure-13: Pipe and Filter Architecture of Source Code Processing

Component: Feature Extraction
Pattern: Pipe and Filter Architecture

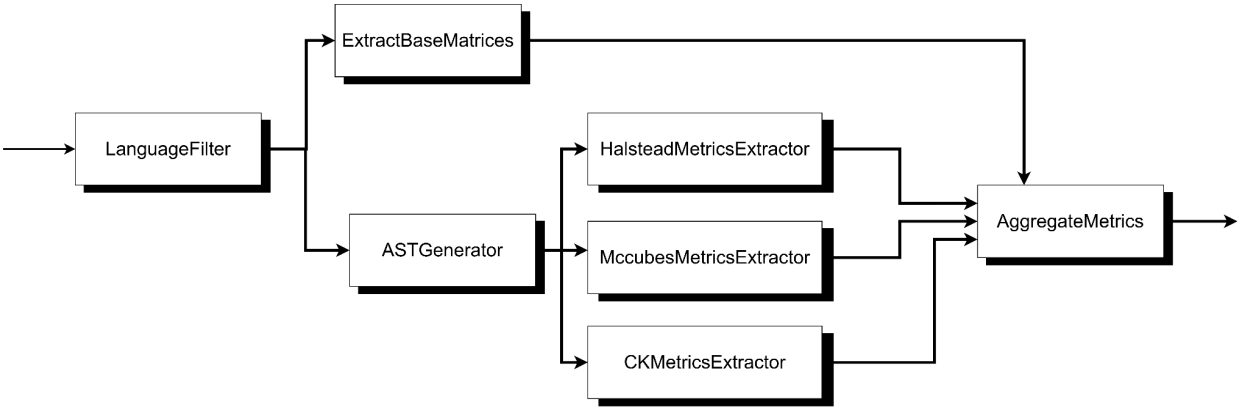


Figure-14: Pipe and Filter Architecture of Feature Extraction

6.4 Mapping Requirements to Software Architecture

The mapping of the derived components with the requirements can be shown as below:

Requirements	Components
Basic interface for code upload and result viewing.	User Interface
Export functionality for defect results.	
File upload options management	
Model accuracy checking interface	

Data storage management	Authentication
User session management	
Access control	
Efficient source code processing	Source Code Processing
ZIP file handling	
Metric extraction (McCabe, Halstead, CK)	Feature Extraction
Code analysis implementation	
Result generation for analysis	
API endpoint integration	
Model training data preparation	Data Alignment
Model adaptability support	
Transfer learning data preparation	
Defect prediction implementation	Classification
Result analysis and reporting	
Accuracy metrics generation	

7. Preliminary Test Plan

In this chapter, a high level description of testing goals and summary of features to be tested are presented.

7.1 High Level Description of Testing Goals

Testing goals for this tool is represented below from a high level-

- Validate accuracy of metrics extraction
- Verify transfer learning model performance
- Ensure reliable defect prediction across projects
- Test file processing and validation
- Verify user data security and history tracking
- Evaluate system performance under varying loads

7.2 Test Cases

Test Case 1: Source Code Upload

Objective: Verify file upload and validation
Input: Zip file containing source code
Steps:

1. Upload zip file through interface
2. Verify file format validation
3. Check extraction process

Expected Result: Successful file validation and extraction.

Test Case 2: Metrics Extraction

Objective: Validate metrics calculation accuracy
Input: Source code files
Steps:

1. Process source files
2. Extract Halstead metrics
3. Calculate McCabe complexity
4. Compute CK metrics

Expected Result: Accurate metrics values matching manual calculations.

Test Case 3: Transfer Learning

Objective: Test model adaptation

Input: Training data and target project code

Steps:

1. Train model on source project
2. Apply transfer learning
3. Validate predictions on target project

Expected Result: Successful knowledge transfer with maintained accuracy.

Test Case 4: Defect Prediction

Objective: Verify prediction accuracy

Input: Processed source code

Steps:

1. Generate predictions
2. Compare with known defects
3. Calculate accuracy metrics

Expected Result: Predictions match known defect patterns.

Test Case 5: User Management

Objective: Test user authentication and history

Input: User credentials and actions

Steps:

1. Register new user
2. Perform multiple submissions
3. Verify history tracking

Expected Result: Accurate user history and data management.

Test Case 6: System Performance

Objective: Evaluate system under load

Input: Multiple concurrent requests

Steps:

1. Submit multiple source files
2. Process parallel requests
3. Monitor response times

Expected Result: System maintains performance under load

Test Case 7: Data Security

Objective: Verify data protection

Input: User submissions and history

Steps:

1. Attempt unauthorized access
2. Verify data encryption
3. Test access controls

Expected Result: Secure data handling and access control

Test Case 8: Result Generation

Objective: Test result formatting and export

Input: Analysis results

Steps:

1. Generate analysis report
2. Export in different formats
3. Verify result accuracy

Expected Result: Correct and readable result reports

Conclusion

In this report, I have thoroughly discussed the technical details necessary to develop "DefectLens: A Software Defect Identifier". The documentation provides a comprehensive overview of the system's architecture, encompassing scenario-based modeling, class-based modeling, and detailed architectural design. I have outlined the system's core functionalities, from source code processing and metrics extraction to the implementation of transfer learning for defect prediction, supported by detailed use cases and class hierarchies. The high-level testing framework ensures robust validation of all system components. While DefectLens effectively addresses automated defect detection through transfer learning, there is room for enhancement through additional language support, improved development environment integration, and advanced visualization capabilities.

With this document, I have strived to present a clear and thorough understanding of the system's workflow and technical implementation, supported by illustrative figures and tables. I hope this report proves valuable for understanding DefectLens's role in improving software quality through automated defect detection.

References:

[1] Hariprasad, T., et al. "Software complexity analysis using halstead metrics." *2017 International Conference on Trends in Electronics and Informatics (ICEI)*. IEEE, 2017.

[2] Harrison, Warren A., and Kenneth I. Magel. "A complexity measure based on nesting level." ACM Sigplan Notices 16.3 (1981): 63-74.

[3] Weiss, Karl, Taghi M. Khoshgoftaar, and DingDing Wang. "A survey of transfer learning." *Journal of Big data* 3 (2016): 1-40.